

Conception et Architecture d'une Application de Messagerie Type WhatsApp

Architecte Logiciel Senior

30 avril 2026

Table des matières

1	Introduction	3
2	Analyse et Conception (UML)	3
2.1	Cas d'Utilisation	3
2.2	Diagramme de Classes	3
3	Architecture Technique (Version 1)	3
3.1	Serveur Central Multithread	3
3.2	Client JavaFX et Module Média	4
3.3	Flux de Messagerie et d'Appels	4
4	Conclusion	4

1 Introduction

Ce document présente l'architecture logicielle et les choix de conception d'une application de messagerie instantanée, conçue pour simuler les fonctionnalités de base de WhatsApp. L'objectif principal de ce projet est de mettre en œuvre les concepts fondamentaux de la programmation orientée objet et de la conception UML, au travers d'une architecture client-serveur robuste. L'application repose sur l'utilisation avancée des threads et des sockets réseaux, permettant une communication fluide en temps réel. L'interface utilisateur est développée à l'aide du framework JavaFX, offrant une expérience moderne et réactive. Les fonctionnalités incluent l'authentification, l'affichage des utilisateurs connectés, l'échange de messages textuels, les appels audio/vidéo, avec une architecture prête pour l'intégration future du partage de fichiers et de photos.

2 Analyse et Conception (UML)

La conception du système a été modélisée en respectant rigoureusement les standards UML.

2.1 Cas d'Utilisation

Le système interagit principalement avec l'Utilisateur final et un acteur système secondaire, la Base de données. Les cas d'utilisation couvrent le cycle de vie de l'interaction utilisateur : de l'inscription et la connexion, jusqu'à l'initiation d'appels multimédias. Une attention particulière a été portée à la modélisation des interactions avec la base de données, représentée par une association de communication standard pour souligner son rôle d'acteur externe sollicité par le système pour la persistance des états et des messages. L'intention future d'ajouter le partage de fichiers est d'ores et déjà anticipée dans cette modélisation.

2.2 Diagramme de Classes

Le diagramme de classes illustre une stricte séparation des préoccupations. L'architecture sépare clairement la couche de présentation (JavaFX), la couche de logique métier (Gestionnaire de discussion) et la couche réseau. L'introduction récente du module média a nécessité l'intégration de classes dédiées telles que `MediaCapture`, `StreamSender` et `StreamReceiver`, qui gèrent la capture matérielle et la transmission des flux audio/vidéo indépendamment de la signalisation TCP.

3 Architecture Technique (Version 1)

3.1 Serveur Central Multithread

Le cœur du système repose sur un serveur centralisé fonctionnant sur un modèle multithread. Chaque client qui se connecte au serveur via un socket TCP se voit attribuer un thread dédié (`ClientHandler`). Cette approche permet au serveur de gérer de multiples connexions simultanées sans blocage. Le serveur est responsable de l'authentification, du maintien de l'état des présences (en ligne/hors ligne), et du routage des messages textuels et des requêtes de signalisation.

3.2 Client JavaFX et Module Média

Côté client, l'interface graphique est gérée par le thread principal JavaFX (**Application Thread**). Pour éviter le gel de l'interface, toutes les opérations d'Entrée/Sortie (E/S) réseaux sont déportées sur des threads d'arrière-plan. Le module média constitue une brique essentielle de cette version : il capture l'audio via le microphone (JavaSound) et la vidéo via la caméra (OpenCV/JavaCV), et orchestre la numérisation des flux pour leur transmission.

3.3 Flux de Messagerie et d'Appels

- **Messagerie (Texte)** : Les messages transitent exclusivement par le serveur central via des connexions TCP fiables. Si le destinataire est hors ligne, le serveur assure la persistance du message dans la base de données PostgreSQL pour une livraison différée.
- **Appels (Audio/Vidéo)** : L'établissement de l'appel suit un modèle hybride. La *signalisation* (demande d'appel, acceptation, échange d'adresses IP) est acheminée de manière fiable via TCP par le serveur central. Une fois l'appel accepté, le *streaming média* s'effectue en Peer-to-Peer (P2P) direct entre les clients via le protocole UDP, minimisant ainsi la latence et soulageant la bande passante du serveur central.

4 Conclusion

La version 1 de cette application de messagerie démontre une architecture évolutive et performante. La séparation stricte entre la signalisation TCP centralisée et le transfert multimédia UDP en P2P garantit une excellente réactivité. La conception orientée objet et les diagrammes UML élaborés assurent une base solide pour les futures itérations, notamment l'intégration du partage de fichiers sécurisé.